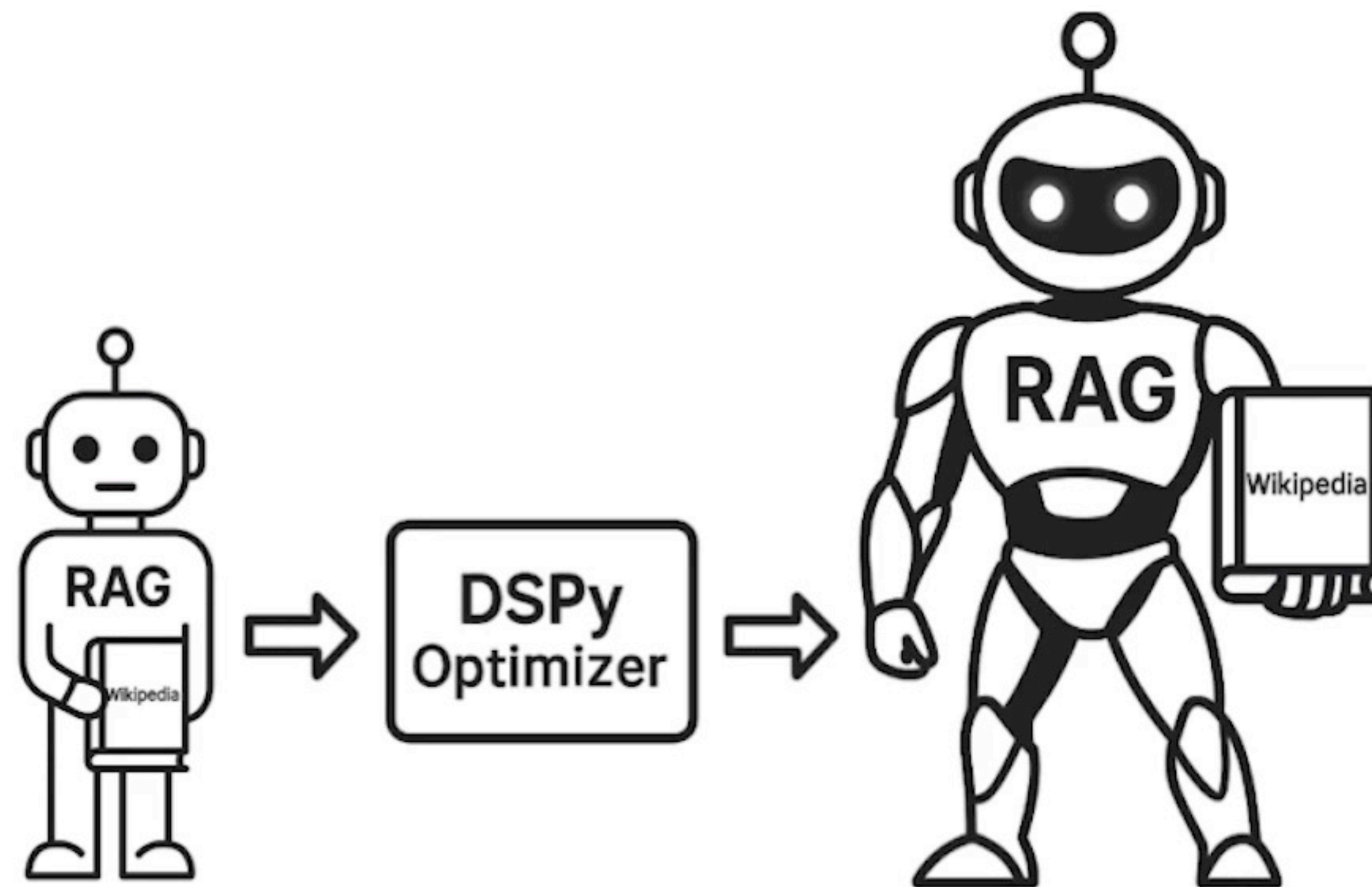


Lab Materials

- Use **DSPy Optimizer**: Automatically improve the quality of a **RAG agent**.
- The data source is **Wikipedia**.



What does optimization mean?

1) Prompt template



- Task description
- Input/output description
- Language specs

2) Few-shot examples



- High-quality examples lead to good response quality
- Tradeoff between cost and quality

3) LM weights



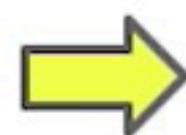
- SFT to let the model learn more about the task
- Preference finetuning (GRPO and else) to have the model align the text

How to Use DSPy Optimizer

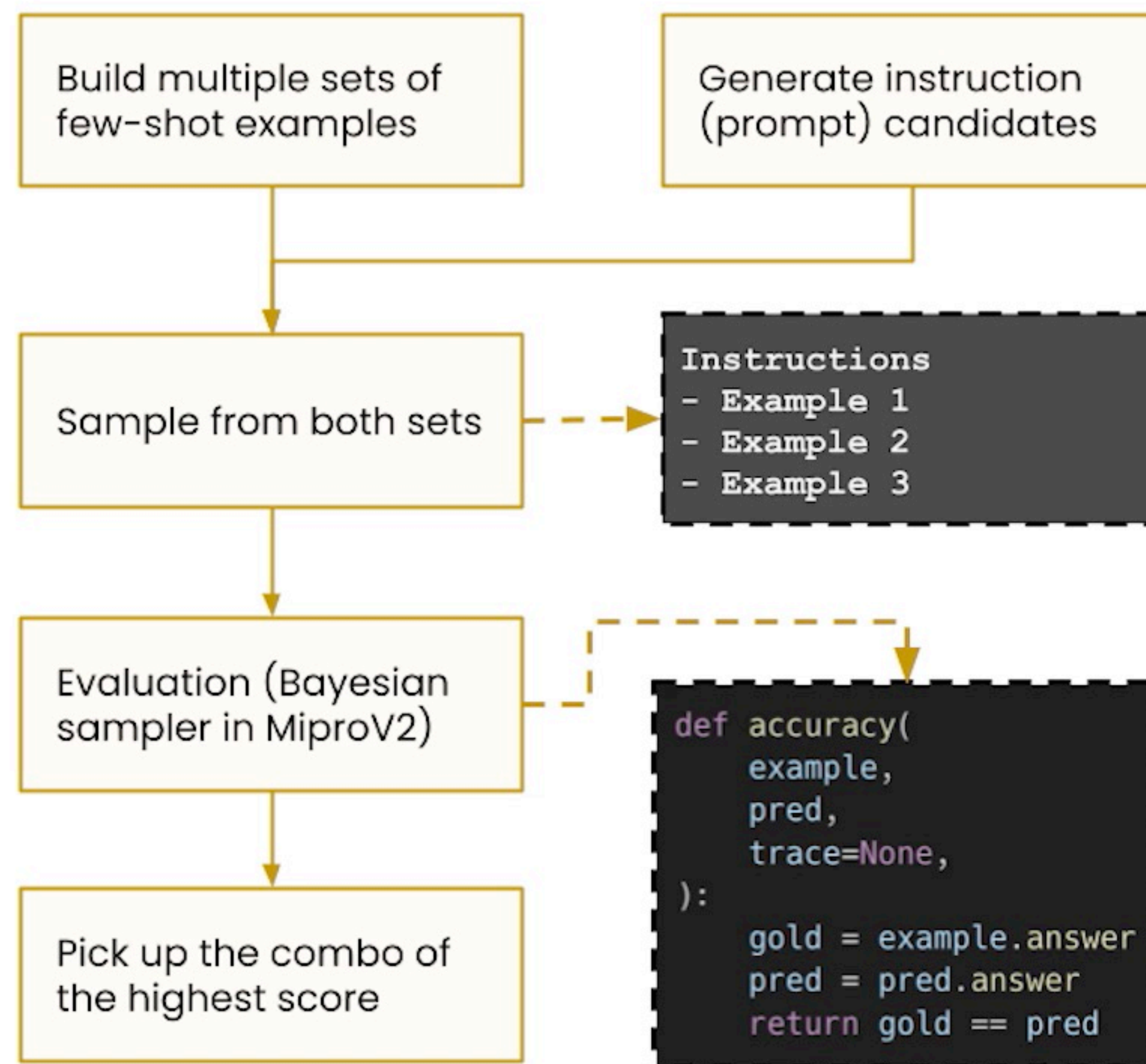
```
def search_wikipedia(query: str) -> list[str]:
    results = dsp.ColBERTv2(
        url="http://20.102.90.50:2017/wiki17_abstracts",
    )(query, k=3)
    return [x["text"] for x in results]

trainset = [
    x.with_inputs("question")
    for x in HotPotQA(
        train_seed=2024,
        train_size=500,
    ).train
]
react = dsp.ReAct(
    "question -> answer",
    tools=[search_wikipedia],
)

tp = dsp.MIPROv2(
    metric=dsp.evaluate.answer_exact_match,
    auto="light",
    num_threads=24,
)
optimized_react = tp.compile(react, trainset=trainset)
```



Prompt Optimization



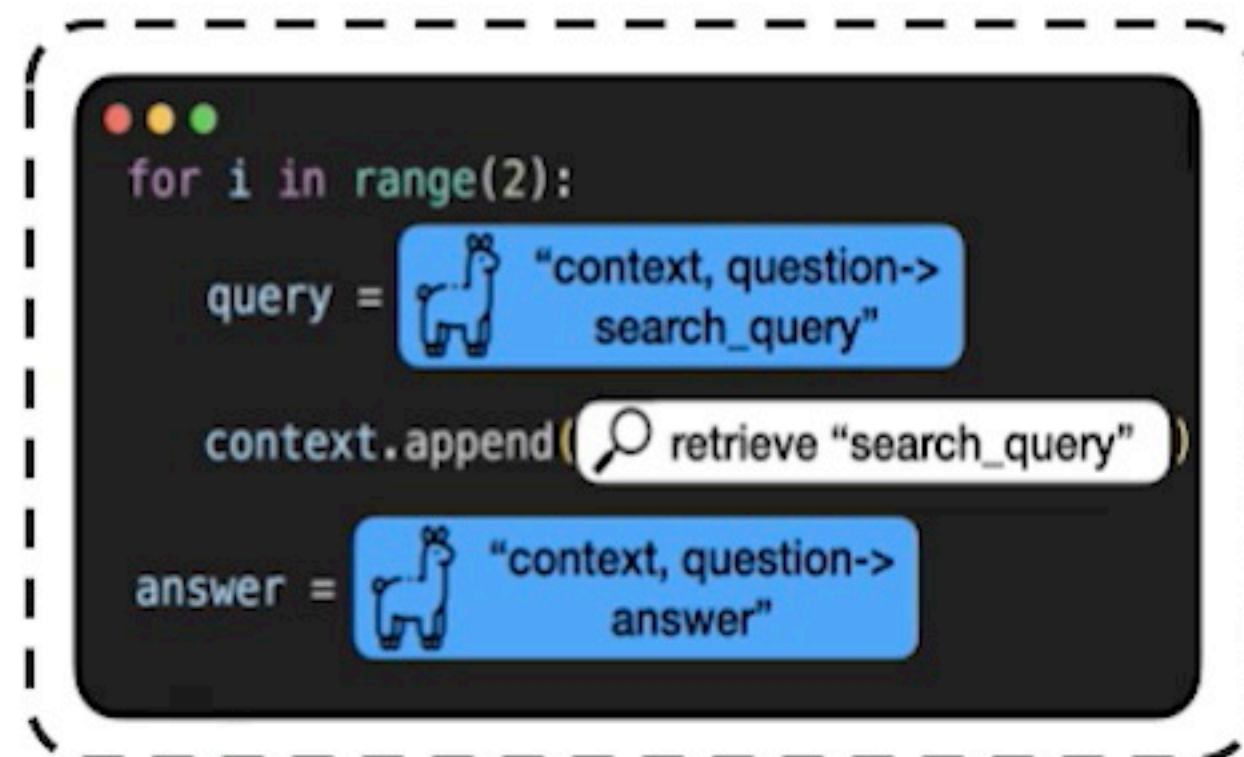
Build Few-shot Examples

Bootstrapping

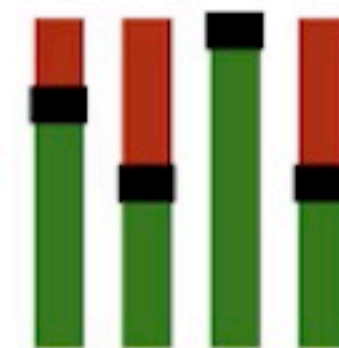
Training Input



LM Program:

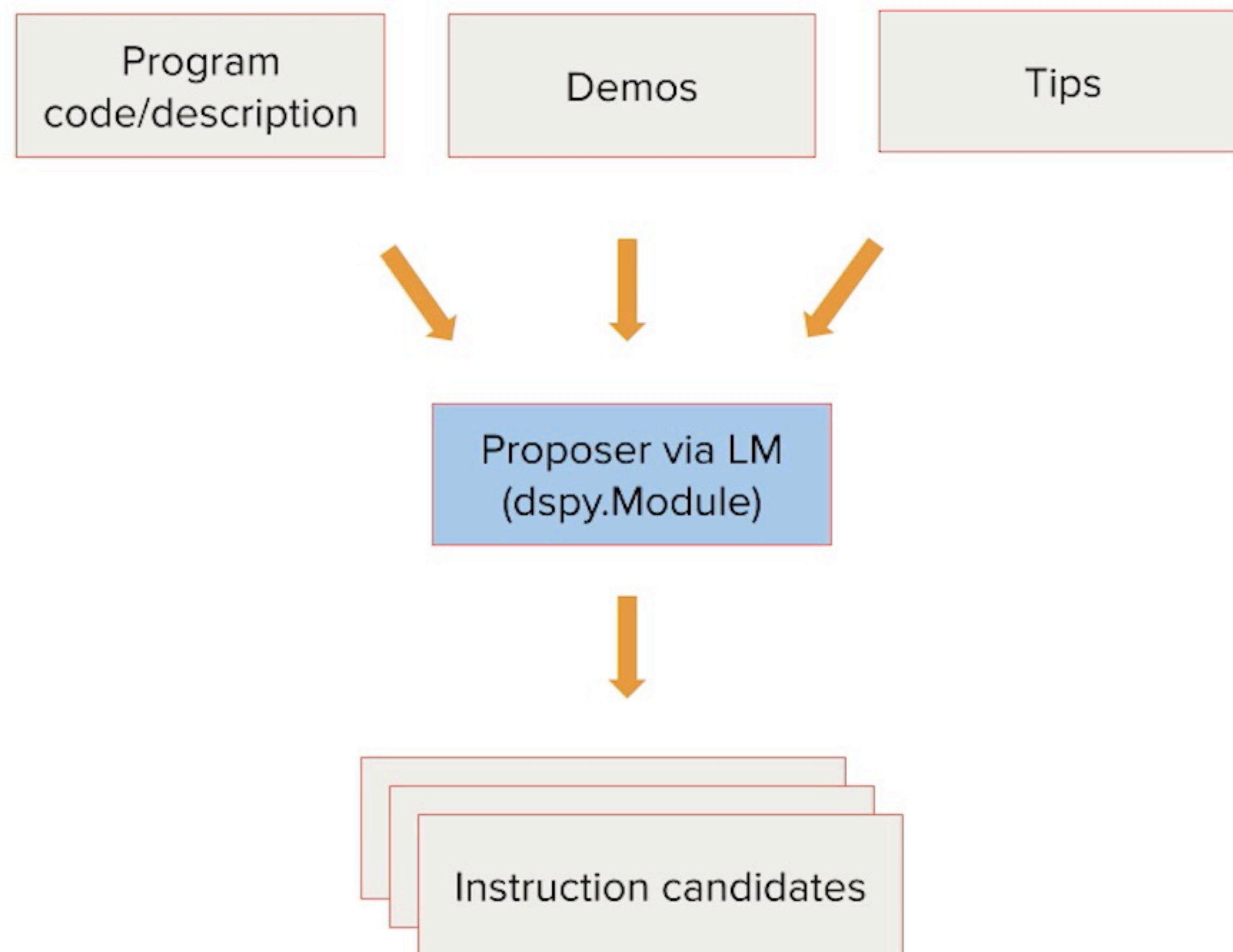


Metric



Good Trace!

Build Instruction Candidates



Optimizer Performance

Optimizer	ScoNe			HotPotQA			HoVer			HotPotQA Cond.			Iris		Iris-Typo		Heart Disease	
	Train	Dev	Test	Train	Dev	Test	Train	Dev	Test	Train	Dev	Test	Train	Test	Train	Test	Train	Test
<i>Instructions only (0-shot)</i>																		
N/A	57.0	56.2	69.1	35.4	31.8	36.1	30.2	30.8	25.3	13.8	10.5	6	46.4	40.9	34.7	32	23.3	26.8
Module-Level OPRO –G	70.0	67.4	76.1	36.0	31.7	36.0	30.0	30.0	25.7	–	–	–	–	–	–	–	–	–
Module-Level OPRO	69.1	67.6	73.5	41.9	36.2	39.0	37.1	38.6	32.5	–	–	–	–	–	–	–	–	–
0-Shot MIPRO	66.3	65.2	71.5	40.2	34.2	36.8	37.7	38.4	33.1	22.6	20.3	14.6	40.8	36.4	56.8	56.7	26.8	25.8
0-Shot MIPRO++	69.0	66.9	75.7	41.5	36.2	39.3	37.1	37.3	32.6	–	–	–	–	–	–	–	–	–
<i>Demonstrations only (Few-shot)</i>																		
Bootstrap RS	74.9	69.6	75.4	48.6	44.0	45.8	42.0	42.0	37.2	16.4	15.0	10.4	95.2	94.1	58.9	58.7	78.4	79.2
Bayesian Bootstrap	75.4	67.4	77.4	49.2	44.8	46.2	44.6	44.7	37.6	–	–	–	–	–	–	–	–	–
<i>Both (Few-shot)</i>																		
MIPRO	74.6	69.8	79.4	49.0	43.9	46.4	44.7	46.7	39.0	28.4	28.1	23.3	98.4	88.6	69.1	68.7	75.2	74.2

Paper: “Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs”

– <https://arxiv.org/pdf/2406.11695>

Track DSPy Optimization via MLflow

```
mlflow.dspy.autolog(  
    log_evals=True,  
    log_compiles=True,  
    log_traces_from_compile=True,  
)
```

MLflow tracks each evaluate call during optimization process.



Let's see all of this in the lab